SOLUCIÓN Y RESULTADOS

- Link Repositorio Propuesta 1: https://github.com/shanccom/IDNP_Repo/tree/main/lab10SincronizacionServicioCompose
- Link Repositorio Propuesta 2: https://github.com/DeniseHuacani/IDNP-LABORATORIO/tree/main/lab10/SincronizacionServicio_Compose

I. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

A continuación se detallan, paso a paso, las instrucciones para implementar un Service que simula la sincronización de la lista de edificios y avisa a la UI mediante un BroadcastReceiver:

Paso 1. Crear un nuevo proyecto

Crear un nuevo proyecto. nombre sugerido para el proyecto: “SincronizacionServicio_Compose”, Package name: com.example.sincronizacionservicio_compose, Language: Kotlin, Minimum SDK: el que venga por defecto en la plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

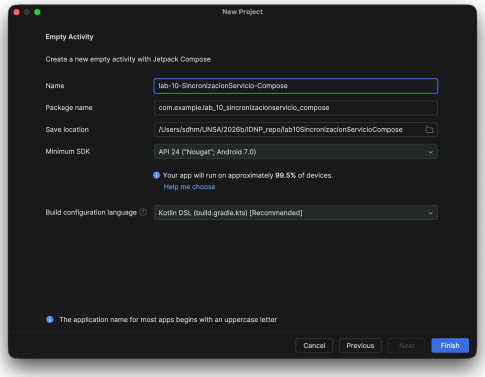


Figure 1: Paso 1: Creación del proyecto SincronizacionServicio_Compose en Android Studio con Kotlin y el SDK mínimo de la plantilla.

Paso 2. Abrir el archivo AndroidManifest.xml y declarar el nuevo Service

Abrir el archivo AndroidManifest.xml y declarar el nuevo Service dentro de la etiqueta `<application>`:

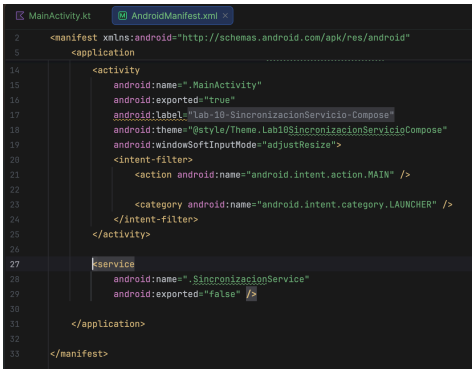


Figure 2: Código (AndroidManifest.xml – fragmento dentro de `<application>`)

Paso 3. Agregar importaciones

Agregar, debajo del package y antes de la clase MainActivity, todas las importaciones que se usarán en este ejercicio:

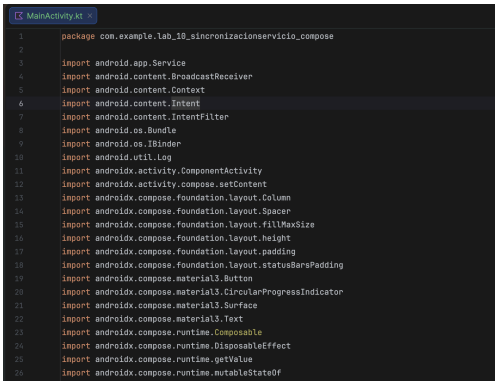


Figure 3: Código (importaciones)

Paso 4. Crear la clase SincronizacionService

Crear la clase SincronizacionService, heredando de Service, y sobrescribir onBind() devolviendo null, ya que no será un Bound Service. Por ahora dejamos pendiente la lógica de sincronización:



Figure 4: Código (SincronizacionService – Paso 4)

Paso 5. Sobrescribir onStartCommand()

Sobrescribir onStartCommand(), lanzando una coroutine en Dispatchers.IO que simule una tarea de sincronización con delay(3000) y calcule un resultado simulado (por ejemplo, un número aleatorio de edificios actualizados):



Figure 5: Código (SincronizacionService – Paso 5)

Paso 6. Construir el Intent y enviar el Broadcast

Al terminar la coroutine, construir un Intent con una acción propia, indicar con setPackage(packageName) que el aviso es solo para esta aplicación, agregar el resultado como extra, enviarlo con sendBroadcast(), registrar un mensaje en el Logcat y llamar a stopSelf() para finalizar el Service. Con esto, SincronizacionService queda completo:



Figure 6: Código (SincronizacionService completo)

Paso 7. Crear el Composable PantallaSincronizacion

Crear el Composable PantallaSincronizacion, obteniendo el contexto actual y una variable de estado para el resultado recibido. Por ahora dejamos el resto del cuerpo vacío; lo iremos completando en los siguientes pasos:



Figure 7: Código (PantallaSincronizacion – Paso 7)

Paso 8. Registrar el BroadcastReceiver con DisposableEffect

Dentro de un DisposableEffect(Unit), crear un BroadcastReceiver que en onReceive() registre un mensaje en el Logcat y actualice resultado con el extra recibido, registrarlo con ContextCompat.registerReceiver() indicando RECEIVER_NOT_EXPORTED, y desregistrarlo en onDispose { }:

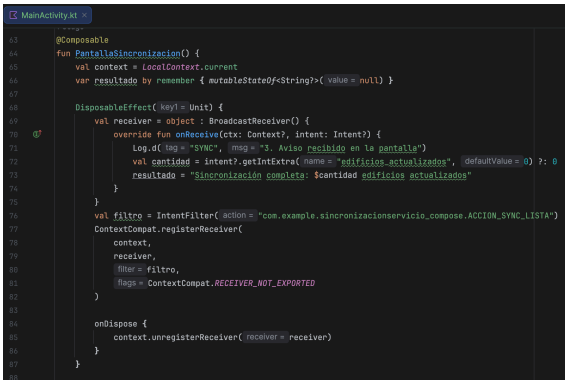


Figure 8: Código (PantallaSincronizacion – Paso 8)

Paso 9. Agregar botón de sincronización e indicador de progreso

Agregar un botón “Sincronizar” que inicie el Service con startService(), y un indicador de progreso que se muestre mientras resultado sea null. El Column usa statusBarsPadding() para que el contenido no quede debajo de la barra de estado. Con esto, PantallaSincronizacion queda completo:



Figure 9: Código (PantallaSincronizacion completo)

Paso 10. Integrar PantallaSincronizacion en MainActivity

Dentro de MainActivity, en el bloque setContent, mostrar PantallaSincronizacion:



Figure 10: Código (MainActivity.kt completo)

Paso 11. Ejecución en emulador y verificación en Logcat

Ejecutar la aplicación en un emulador, pulsar “Sincronizar” una vez y verificar en Logcat (filtrando con tag:SYNC) que aparece el mensaje “1. Servicio iniciado”, mientras la UI muestra el indicador de progreso.

Pruebas de Integrante 1

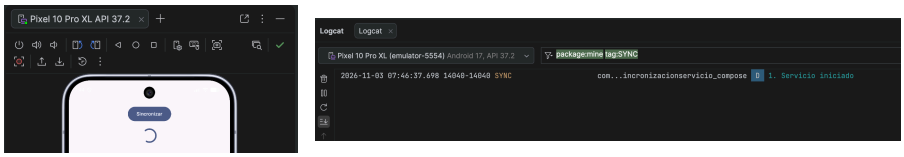


Figure 11: Paso 11: Ejecución de la aplicación en el emulador con el indicador de progreso activo y verificación en Logcat (“1. Servicio iniciado”).

Pruebas de Integrante 2



Figure 12: Paso 11 (Integrante 2): Ejecución de la aplicación en el emulador con el indicador de progreso activo y verificación en Logcat (“1. Servicio iniciado”).

Paso 12. Confirmar sincronización y recepción del Broadcast

Esperar a que termine la coroutine (unos 3 segundos) y confirmar que en Logcat aparecen “2. Aviso enviado” y “3. Aviso recibido en la pantalla”, y que la UI reemplaza el indicador por el mensaje recibido mediante el BroadcastReceiver.

Pruebas de Integrante 1

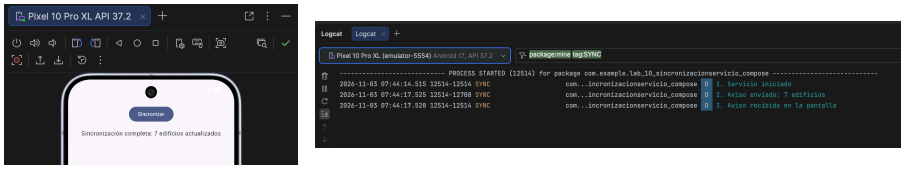


Figure 13: Paso 12: Confirmación en Logcat de la finalización de la coroutine y actualización reactiva de la UI mediante el BroadcastReceiver.

Pruebas de Integrante 2

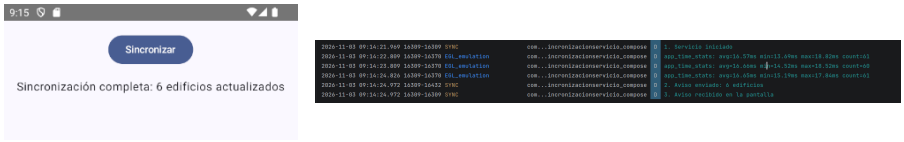


Figure 14: Paso 12 (Integrante 2): Confirmación en Logcat de la finalización de la coroutine y actualización reactiva de la UI mediante el BroadcastReceiver.

EJERCICIOS/PROBLEMAS PROPUESTOS

Amplía el ejercicio resuelto para dar más control al usuario sobre la sincronización:

Propuesta 1

1. Agrega un botón “Detener sincronización” que llame a context.stopService(Intent(context, SincronizacionService::class.java)) mientras la tarea está en curso.

Para detener la sincronización en ejecución, se extendió la interfaz en `PantallaSincronizacion()` dentro de `MainActivity.kt`.

Cambios Realizados

Archivo modificado: MainActivity.kt

1. **Nuevo estado `enCurso`:** Se incorporó `var enCurso by remember { mutableStateOf(false) }` para modelar de forma reactiva si el servicio de sincronización está activo.
2. **Inicio de sincronización:** Al pulsar **"Sincronizar"**, se reinicia `resultado = null`, se asigna `enCurso = true` y se invoca `context.startService(...)`.
3. **Finalización por Broadcast:** En el `BroadcastReceiver`, al recibir la confirmación de término de sincronización, se asigna `enCurso = false` y se actualiza el mensaje con los edificios procesados.
4. **Botón "Detener sincronización":** Se añadió un botón condicional visible únicamente mientras `enCurso == true`. Al hacer clic, invoca `context.stopService(Intent(context, SincronizacionService::class.java))` y actualiza el estado local a `enCurso = false`.
5. **Indicador visual reactivo:** El componente `CircularProgressIndicator()` se gobierna ahora según el valor de `enCurso` (en lugar de evaluar `resultado == null`).
6. **Alcance de modificaciones:** No se requirió alterar el servicio `SincronizacionService`, el archivo `AndroidManifest.xml` ni las importaciones previas.

Código Implementado en `PantallaSincronizacion()`

```
// 1. Estado reactivo para rastrear si la sincronización está activa
var enCurso by remember { mutableStateOf(false) }

// En el BroadcastReceiver:
val receiver = object : BroadcastReceiver() {
    override fun onReceive(c: Context?, intent: Intent?) {
        val cantidad = intent?.getIntExtra("CANTIDAD", 0) ?: 0
        resultado = "Sincronización completa: $cantidad edificios actualizados"
        // 3. Se desactiva el estado al recibir el aviso de finalización
        enCurso = false
    }
}

// Dentro del Column:
Button(onClick = {
    resultado = null
    // 2. Se activa el estado al iniciar el servicio
    enCurso = true
    context.startService(Intent(context, SincronizacionService::class.java))
}) {
    Text("Sincronizar")
}

// 4. Botón visible únicamente mientras la tarea está en curso
if (enCurso) {
    Spacer(modifier = Modifier.height(16.dp))
    Button(onClick = {
        context.stopService(Intent(context, SincronizacionService::class.java))
        enCurso = false
    }) {
        Text("Detener sincronización")
    }
}

Spacer(modifier = Modifier.height(16.dp))

// 5. El indicador de carga o el resultado se renderizan según enCurso
if (enCurso) {
    CircularProgressIndicator()
} else {
    Text(resultado ?: "")
}
```

Evidencias

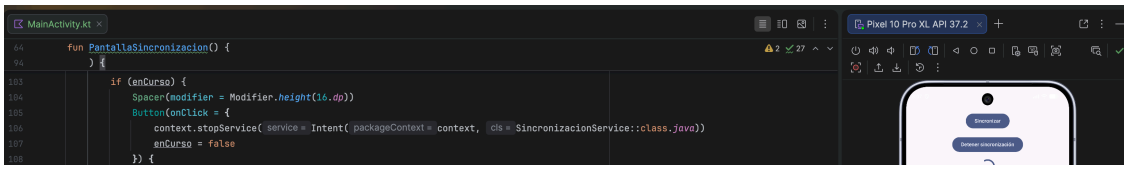


Figure 15: Implementación en `MainActivity.kt` y visualización simultánea de la interfaz en Android Studio.

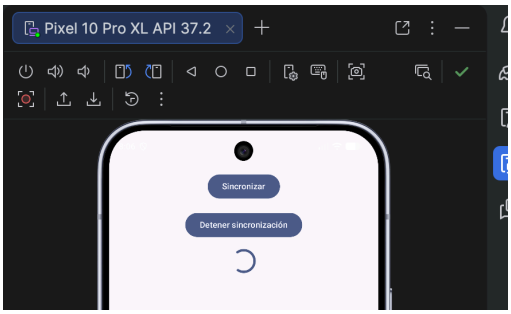


Figure 16: Emulador ejecutando la sincronización, mostrando el botón “Detener sincronización” y el indicador de progreso activo.

2. Modifica el Service para que, en lugar de un resultado simulado, recorra la lista de edificios persistida en Room (Laboratorio 09) y “sincronice” (por ejemplo, marque como actualizados) los que cumplan una condición, enviando el conteo real por el broadcast.

Se sustituyó la sincronización simulada por una persistencia real con Room (reutilizando el esquema del Laboratorio 09), sincronizando únicamente aquellos edificios con `sincronizado == false` y reportando métricas reales a la UI.

Archivos Configurados y Modificados

Archivo	Cambio Realizado
`EdificiosRoom.kt`	Nueva entidad `EdificioEntity` (con flags `sincronizado` y `actualizadoEn`), `EdificioDao`, `AppDatabase`, `DatabaseProvider` y función de sembrado inicial.
`MainActivity.kt`	El servicio consulta Room, actualiza pendientes en bucle con delay, cancela su `Job` en `onDestroy()` y emite el conteo real; la UI muestra “N de M”.
`build.gradle.kts` / `app`	Configuración de KSP (`2.3.6`) y dependencias de Room (`2.7.2`: runtime, ktx y compiler).
`libs.versions.toml`	Declaración de dependencias y versiones de Room y KSP en el catálogo de versiones.

1. Estructura de Room (`EdificiosRoom.kt`)

Se definió la entidad y el DAO para gestionar la condición de sincronización:

```
@Entity
data class EdificioEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val nombre: String,
    val categoria: String,
    val sincronizado: Boolean = false,   // <-- estado de sincronización
    val actualizadoEn: Long? = null
)

@Dao
interface EdificioDao {
    @Insert
    suspend fun insertarEdificio(edificio: EdificioEntity): Long

    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerTodos(): List<EdificioEntity>

    // Condición: edificios pendientes de sincronizar
    @Query("SELECT * FROM EdificioEntity WHERE sincronizado = 0")
    suspend fun obtenerPendientes(): List<EdificioEntity>

    @Query("UPDATE EdificioEntity SET sincronizado = 1, actualizadoEn = :momento WHERE id = :id")
```

```
    suspend fun marcarSincronizado(id: Int, momento: Long)
}
```

Incluye además `AppDatabase`, `DatabaseProvider` (siguiendo el patrón del Lab 09) y la función `sembrarEdificiosIniciales(dao)`, la cual inserta 4 edificios de ejemplo (los 2 del Lab 09 más Pabellón de Aulas y Comedor) únicamente si la base está vacía, asegurando datos de prueba reproducibles.

2. Consulta Real y Cancelación en `SincronizacionService`

El servicio consulta Room en un hilo IO, actualiza cada registro pendiente y vincula la corrutina a un `Job` para corte inmediato al detenerse:

```
private var trabajo: Job? = null

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
    trabajo = CoroutineScope(Dispatchers.IO).launch {
        val dao = DatabaseProvider.obtener(applicationContext).edificioDao()
        sembrarEdificiosIniciales(dao)

        val pendientes = dao.obtenerPendientes()   // Condición real
        var actualizados = 0

        for (edificio in pendientes) {
            delay(500) // Simula la sincronización remota
            dao.marcarSincronizado(edificio.id, System.currentTimeMillis())
            actualizados++
        }

        val resultado =
Intent("com.example.lab_10_sincronizacionservicio_compose.ACCION_SYNC_LISTA").apply {
            setPackage(packageName)
            putExtra("edificios_actualizados", actualizados) // Conteo real
            putExtra("edificios_revisados", pendientes.size)
        }
        sendBroadcast(resultado)
        stopSelf()
    }
    return START_NOT_STICKY
}

override fun onDestroy() {
    trabajo?.cancel()   // "Detener sincronización" cancela la corrutina activamente
    super.onDestroy()
}
```

3. Actualización de la UI con Conteo Real

En el `BroadcastReceiver` de `PantallaSincronizacion`, se procesan los datos reales reportados por el servicio:

```
val revisados = intent?.getIntExtra("edificios_revisados", 0) ?: 0
val cantidad = intent?.getIntExtra("edificios_actualizados", 0) ?: 0

resultado = if (revisados == 0) {
    "Sin edificios pendientes de sincronizar"
} else {
    "Sincronización completa: $cantidad de $revisados edificios actualizados"
}
```

Evidencias

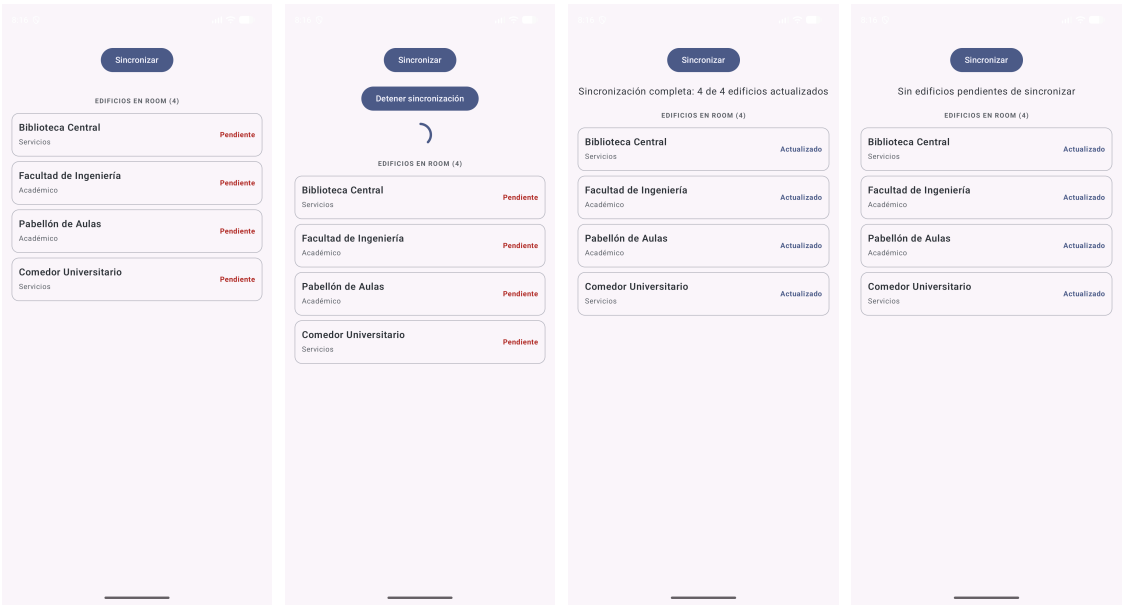


Figure 17: Ciclo de vida y sincronización real con Room: (a) 4 edificios pendientes, (b) sincronización en curso con indicador y botón de detención, (c) primera sincronización (4 de 4 actualizados), y (d) segunda sincronización sin registros pendientes.

3. Registra un segundo BroadcastReceiver que escuche una acción distinta (por ejemplo “com.tuapp.ACCION_SYNC_ERROR”) y simula un caso de error en el Service (por ejemplo, con una probabilidad aleatoria), mostrando un mensaje distinto en la UI.

Se introdujo la acción de difusión `ACCION_SYNC_ERROR` para simular y canalizar fallos de sincronización. Antes de consultar Room, el servicio evalúa una probabilidad del 30 % (`Random.nextInt(100) < 30`); de cumplirse, emite el broadcast con el extra `mensaje_error`, escribe en Logcat y ejecuta `stopSelf()` sin modificar la base de datos. En la UI, un segundo `BroadcastReceiver` captura la señal, apaga el indicador de carga y muestra el mensaje de error.

Cambios Realizados

Archivo modificado: MainActivity.kt

1. **Acción adicional:** Definición de `ACCION_SYNC_ERROR` junto a `ACCION_SYNC_LISTA`.
2. **Fallo en segundo plano (30 %):** Al dispararse, envía el broadcast con `mensaje_error` y sale con `stopSelf()`.
3. **Segundo receptor reactivo:** `receiverError` (`RECEIVER_NOT_EXPORTED`) actualiza `resultado` y apaga `enCurso`.
4. **Ciclo de vida y consistencia:** Ambos receptores se desregistran en `onDispose` y los 4 edificios siguen **Pendiente**.

Código Implementado

```
// 1. Acciones de difusión
private const val ACCION_SYNC_LISTA = "com.example.sincronizacionservicio_compose.ACCION_SYNC_LISTA"
private const val ACCION_SYNC_ERROR = "com.example.sincronizacionservicio_compose.ACCION_SYNC_ERROR"
// 2. Simulación de fallo aleatorio (30%) en el Service
delay(800)
if (Random.nextInt(100) < 30) {
    val error = Intent(ACCION_SYNC_ERROR).apply {
        setPackage(packageName)
        putExtra("mensaje_error", "No se pudo conectar con el servidor de sincronización")
    }
    sendBroadcast(error)
    Log.d("SYNC", "2b. Error de sincronización notificado")
    stopSelf()
    return@launch
}
// 3. Segundo BroadcastReceiver en PantallaSincronizacion
val receiverError = object : BroadcastReceiver() {
```

```
    override fun onReceive(ctx: Context?, intent: Intent?) {
        Log.d("SYNC", "3b. Error recibido en la pantalla")
        val mensaje = intent?.getStringExtra("mensaje_error") ?: "Error desconocido"
        resultado = "Error de sincronización: $mensaje"
        enCurso = false
    }
}
ContextCompat.registerReceiver(
    context, receiverError, IntentFilter(ACCION_SYNC_ERROR),
    ContextCompat.RECEIVER_NOT_EXPORTED
)
// 4. Desregistro de ambos receptores en onDispose
onDispose {
    context.unregisterReceiver(receiver)
    context.unregisterReceiver(receiverError)
}
```

Evidencias de Ejecución



Figure 18: Registro en Logcat mostrando la secuencia de fallo: servicio iniciado (`1`), notificación de error desde el servicio (`2b`) y recepción en la pantalla (`3b`).



Figure 19: Emulador reflejando el mensaje de error capturado por el segundo receptor; nótese que los edificios permanecen en estado “Pendiente”.

4. Investiga qué son los WorkManager de Android y explica en el informe en qué casos convendría usar WorkManager en lugar de un Service iniciado manualmente para esta misma tarea de sincronización.

WorkManager (`androidx.work`) es la biblioteca de Android Jetpack para ejecutar **trabajo en segundo plano diferible y garantizado**. A diferencia de un `Service` manual (que corre en la memoria RAM y muere al cerrarse la app), WorkManager persiste las tareas en SQLite interno, reprogramándolas incluso tras reiniciar el dispositivo o bajo políticas de ahorro energético (**Doze Mode**).

Criterio	Service Manual (`startService`)	WorkManager (`androidx.work`)
Disparo y Momento	Manual e inmediato desde la UI (tiempo real)	Diferible; programado según condiciones del sistema
Persistencia / Reinicio	Efímero; se pierde al cerrar la app o reiniciar	Garantizado; persiste en SQLite interno tras reinicios
Restricciones	Verificación manual imperativa en código	Declarativas (`Constraints`: red Wi-Fi, batería, carga)
Gestión de Fallos	Manual; requiere lógica de reintento propia	Reintentos automáticos con política de **backoff**
Propósito Idóneo	Interacción en pantalla con respuesta inmediata	Sincronización periódica o diferida desatendida

Criterio de decisión para la sincronización de edificios (Lab 10):

- **¿Cuándo convendría WorkManager?** Si Room debe actualizarse de forma periódica (p. ej., cada 12 h sin abrir la app), con red Wi-Fi obligatoria (`NetworkType.UNMETERED`), reintentos automáticos con **backoff** y garantía de ejecución tras reinicios.
- **¿Por qué se mantiene el Service en el laboratorio?** Porque la tarea es interactiva: el usuario pulsa **“Sincronizar”** esperando feedback visual instantáneo (spinner, progreso y botón **“Detener”**). WorkManager es diferible y no garantiza ejecución inmediata en la UI.



Figure 20: Service manual (ejecución efímera en RAM) vs. WorkManager (persistencia en SQLite con ejecución condicionada).

Implementación de referencia con Room:

```
// Worker con Room, reintentos con backoff y programación periódica con restricciones
class SyncWorker(ctx: Context, p: WorkerParameters) : CoroutineWorker(ctx, p) {
    override suspend fun doWork(): Result = try {
        val dao = DatabaseProvider.obtener(applicationContext).edificioDao()
        dao.obtenerPendientes().forEach { dao.marcarSincronizado(it.id, System.currentTimeMillis()) }
        Result.success()
    } catch (e: Exception) { Result.retry() }
}
val req = PeriodicWorkRequestBuilder<SyncWorker>(6, TimeUnit.HOURS)
   .setConstraints(Constraints.Builder().setRequiredNetworkType(NetworkType.CONNECTED).build()).build()
WorkManager.getInstance(context).enqueueUniquePeriodicWork("sync_edificios",
ExistingPeriodicWorkPolicy.KEEP, req)
```

Propuesta 2

1. Agrega un botón “Detener sincronización” que llame a context.stopService(Intent(context, SincronizacionService::class.java)) mientras la tarea está en curso.

Agregué el botón «Detener sincronización», que llama a `context.stopService(...)`. Al probarlo vi que solo eso no basta: la tarea seguía corriendo y a los 3 segundos igual se enviaba el aviso. Por eso le di al Service su propio `scope` y lo cancelo en `onDestroy()`, que es lo que Android ejecuta cuando el Service se detiene.

En la pantalla añadí el estado `sincronizando`: mientras la tarea corre, «Sincronizar» se desactiva y «Detener sincronización» se activa. Así no se pueden lanzar dos tareas a la vez, y el indicador de progreso solo se ve cuando realmente se está sincronizando. La acción del broadcast quedó como constante (`ACCION_SYNC_LISTA`) para no repetirla.

Código agregado (`MainActivity.kt`):

```
const val ACCION_SYNC_LISTA = "com.example.sincronizacionservicio_compose.ACCION_SYNC_LISTA"

class SincronizacionService : Service() {

    // scope propio del servicio, para poder cancelar la tarea si lo detienen
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.IO)

    // onStartCommand ahora usa scope.launch { ... } en lugar de CoroutineScope(...).launch

    // se ejecuta al detener el servicio (con stopService o con stopSelf)
    override fun onDestroy() {
        scope.cancel()
        Log.d("SYNC", "4. Servicio destruido (onDestroy)")
        super.onDestroy()
    }
}

// En PantallaSincronizacion
var sincronizando by remember { mutableStateOf(false) }

Button(
    onClick = {
        resultado = null
        sincronizando = true
        context.startService(Intent(context, SincronizacionService::class.java))
    },
    enabled = !sincronizando
```

```
) { Text("Sincronizar") }

Button(
    onClick = {
        Log.d("SYNC", "Detención manual solicitada")
        context.stopService(Intent(context, SincronizacionService::class.java))
        sincronizando = false
        resultado = "Sincronización detenida por el usuario"
    },
    enabled = sincronizando
) { Text("Detener sincronización") }

if (sincronizando) CircularProgressIndicator()
else Text(resultado ?: "Pulsa «Sincronizar» para comenzar")
```



Figure 21: Paso 1 (a): pantalla inicial con el botón «Detener» deshabilitado.

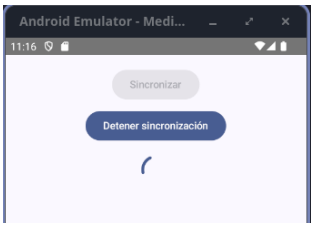


Figure 22: Paso 1 (b): sincronizando; ahora el botón «Detener» está habilitado.



Figure 23: Paso 1 (c): mensaje después de detener manualmente la sincronización.

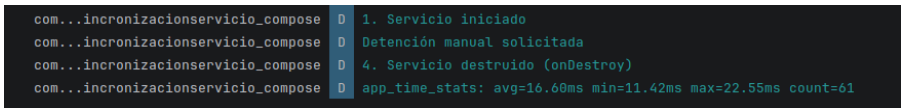


Figure 24: Paso 1 (d): Logcat con «Detención manual solicitada» y «4. Servicio destruido»; no aparece «2. Aviso enviado» porque la tarea se canceló.

2. Modifica el Service para que, en lugar de un resultado simulado, recorra la lista de edificios persistida en Room (Laboratorio 09) y “sincronice” (por ejemplo, marque como actualizados) los que cumplan una condición, enviando el conteo real por el broadcast.

Repetí la configuración de Gradle del Laboratorio 09. Creé la entidad `EdificioEntity` con un campo nuevo, `sincronizado`, que empieza en `false`, y un Dao para insertar, actualizar y leer. La lectura tiene dos versiones: una que devuelve la lista (para el Service) y otra que devuelve un `Flow` (para que la pantalla se actualice sola).

Al abrir la app se insertan 7 edificios de muestra y se muestran en una lista como «Pendiente» o «Sincronizado»; al inicio todos están pendientes.

Código agregado (`gradle/libs.versions.toml`):

```
[versions]
ksp = "2.3.6"

[plugins]
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

Código agregado (`build.gradle.kts` del proyecto y del módulo app):

```
// build.gradle.kts del proyecto, dentro de plugins { }
alias(libs.plugins.ksp) apply false

// build.gradle.kts del módulo app, dentro de plugins { }
alias(libs.plugins.ksp)

// build.gradle.kts del módulo app, dentro de dependencies { }
implementation("androidx.room:room-runtime:2.7.2")
implementation("androidx.room:room-ktx:2.7.2")
ksp("androidx.room:room-compiler:2.7.2")
```

Código agregado (`MainActivity.kt`):

```
@Entity
data class EdificioEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String,
    val categoria: String,
    val sincronizado: Boolean = false
)

@Dao
interface EdificioDao {
    @Insert suspend fun insertar(edificio: EdificioEntity)
    @Update suspend fun actualizar(edificio: EdificioEntity)

    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerTodos(): List<EdificioEntity>

    @Query("SELECT * FROM EdificioEntity")
    fun observarTodos(): Flow<List<EdificioEntity>>
}

@Database(entities = [EdificioEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun edificioDao(): EdificioDao

    companion object {
        @Volatile private var instancia: AppDatabase? = null

        // una sola instancia, compartida por la pantalla y por el Service
        fun obtener(context: Context): AppDatabase =
            instancia ?: synchronized(this) {
                instancia ?: Room.databaseBuilder(
                    context.applicationContext, AppDatabase::class.java, "sincronizacion.db"
                ).build().also { instancia = it }
            }
    }
}

// En PantallaSincronizacion
val dao = remember { AppDatabase.obtener(context).edificioDao() }
val edificiosFlow = remember { dao.observarTodos() }
val edificios by edificiosFlow.collectAsState(initial = emptyList())

LaunchedEffect(Unit) {
    if (dao.obtenerTodos().isEmpty()) {
        // se insertan 7 edificios de muestra
    }
}

LazyColumn(modifier = Modifier.weight(1f).fillMaxWidth()) {
    items(edificios, key = { it.id }) { edificio -> FilaEdificio(edificio) }
}
```



Figure 25: Paso 2 (a): lista de edificios guardados en Room, todos «Pendiente».

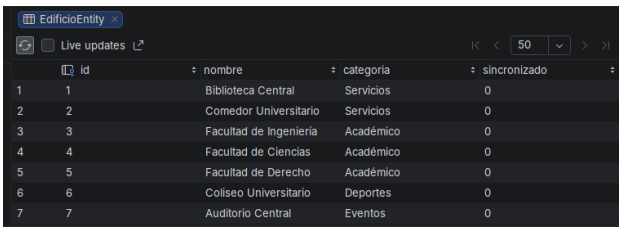


Figure 26: Paso 2 (b): Database Inspector con la tabla `EdificioEntity` y su columna `sincronizado`.

Después cambié el Service para que sincronice de verdad. Tras la espera de 3 segundos, recorre los edificios y marca como sincronizados los de categoría «Académico», contando cuántos son. Ese conteo real es el que se envía en el broadcast, en lugar del número al azar.

Código agregado (`SincronizacionService` en `MainActivity.kt`):

```
scope.launch {
    delay(3000)

    // se recorre la lista guardada en Room
    val dao = AppDatabase.obtener(applicationContext).edificioDao()
    var actualizados = 0
    dao.obtenerTodos().forEach { edificio ->
        if (edificio.categoria == "Académico") {
            dao.actualizar(edificio.copy(sincronizado = true))
            actualizados++
        }
    }

    val resultado = Intent(ACCION_SYNC_LISTA)
    resultado.setPackage(packageName)
    resultado.putExtra("edificios_actualizados", actualizados) // conteo real
    sendBroadcast(resultado)
    Log.d("SYNC", "2. Aviso enviado: $actualizados edificios")

    stopSelf()
}
```



Figure 27: Paso 2 (c): sincronización en curso con el indicador de progreso.

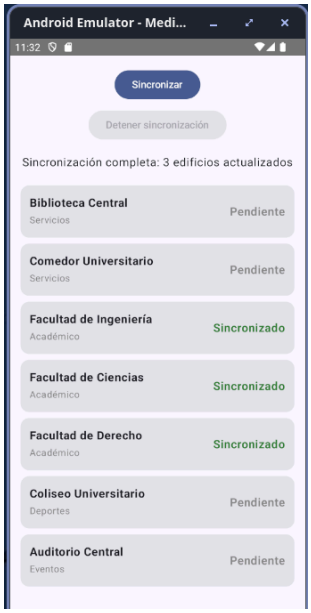


Figure 28: Paso 2 (d): resultado con el conteo real; los edificios «Académico» pasan a «Sincronizado».

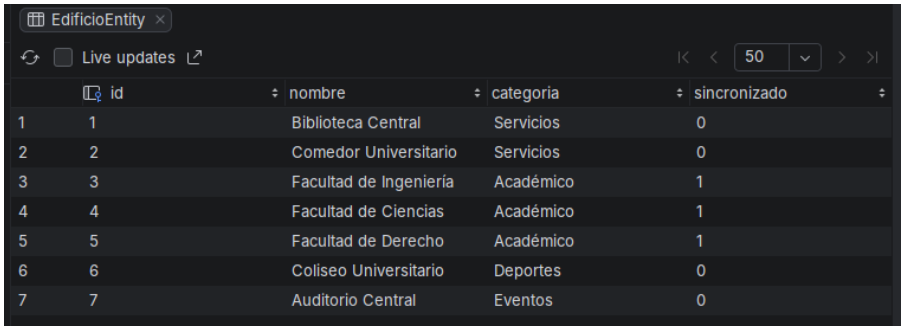


Figure 29: Paso 2 (f): Logcat con los mensajes 1, 2 y 3; el aviso informa el conteo real.

3. Registra un segundo BroadcastReceiver que escuche una acción distinta (por ejemplo “com.tuapp.ACCION_SYNC_ERROR”) y simula un caso de error en el Service (por ejemplo, con una probabilidad aleatoria), mostrando un mensaje distinto en la UI.

Definí una segunda acción, `ACCION_SYNC_ERROR`. En la pantalla registré un segundo receiver, con su propia acción, y lo desregistro junto con el primero. Al recibir el aviso de error, muestra el mensaje con el color de error del tema, para distinguirlo del mensaje de éxito. Para tomar la captura del error cambié temporalmente `(1..3)` por `(1..1)` y luego lo devolví a como estaba.

Código agregado (`MainActivity.kt`):

```
const val ACCION_SYNC_ERROR = "com.example.sincronizacionservicio_compose.ACCION_SYNC_ERROR"

// En el Service, después del delay(3000)
if ((1..3).random() == 1) {
    val error = Intent(ACCION_SYNC_ERROR)
    error.setPackage(packageName)
    error.putExtra("motivo", "No se pudo conectar con el servidor (simulado)")
    sendBroadcast(error)
    Log.d("SYNC", "2. Error simulado: aviso de error enviado")
    stopSelf()
    return@launch
}

// En PantallaSincronizacion
var esError by remember { mutableStateOf(false) }

// dentro del DisposableEffect: segundo receiver
val receiverError = object : BroadcastReceiver() {
    override fun onReceive(ctx: Context?, intent: Intent?) {
        Log.d("SYNC", "3. Aviso de error recibido en la pantalla")
        val motivo = intent?.getStringExtra("motivo") ?: "Error desconocido"
        resultado = "Error de sincronización: $motivo"
        esError = true
        sincronizando = false
    }
}
ContextCompat.registerReceiver(
    context, receiverError, IntentFilter(ACCION_SYNC_ERROR),
    ContextCompat.RECEIVER_NOT_EXPORTED
)
onDispose {
    context.unregisterReceiver(receiverOk)
    context.unregisterReceiver(receiverError)
}

// El mensaje de error se muestra con otro color
Text(
    text = resultado ?: "Pulsa «Sincronizar» para comenzar",
    color = if (esError) MaterialTheme.colorScheme.error else Color.Unspecified
)
```



Figure 30: Paso 3 (a): sincronización exitosa con su mensaje normal.

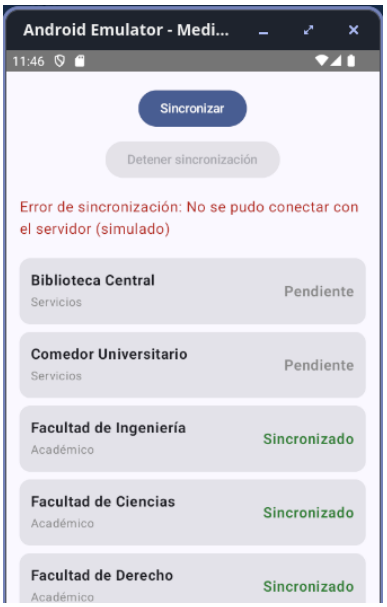


Figure 31: Paso 3 (b): caso de error; la UI muestra otro mensaje y con otro color.

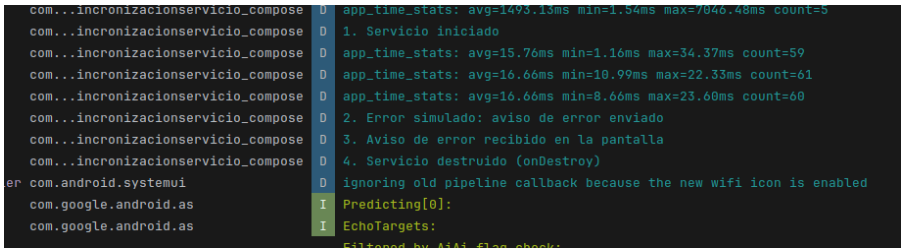


Figure 32: Paso 3 (c): Logcat con «Error simulado» y «Aviso de error recibido en la pantalla».

4: Investiga qué son los WorkManager de Android y explica en el informe en qué casos convendría usar WorkManager en lugar de un Service iniciado manualmente para esta misma tarea de sincronización.

¿Qué es? Según la documentación de Android, WorkManager es la solución recomendada para el trabajo persistente, es decir, sigue programado aunque la app se cierre o el teléfono se reinicie. Se define la tarea y sus condiciones (por ejemplo, tener internet) y se ejecuta cuando esas condiciones se cumplen.

¿Cuándo usarlo en lugar de un Service?

- Cuando debe terminar aunque el usuario cierre la app: Android indica que, cuando una app pasa a segundo plano, el sistema detiene sus Services en segundo plano pasados unos minutos;
- Cuando debe repetirse sola, por ejemplo cada hora: WorkManager permite trabajo periódico ;
- Cuando solo debe correr con ciertas condiciones, como tener conexión: se declaran como restricciones ;
- Cuando hay que reintentar si falla: ofrece políticas de reintento, con espera exponencial configurable.

¿Cuando usar un Service? Sigue siendo mejor cuando la tarea es inmediata y el usuario la espera, como el botón «Sincronizar» de este laboratorio. Si la tarea es larga y el usuario la nota, la documentación recomienda un Service en primer plano, con notificación.

Como referencia, así se programaría la sincronización periódica con WorkManager, según la documentación de Android:

```
class SincronizacionWorker(ctx: Context, params: WorkerParameters) :
    CoroutineWorker(ctx, params) {
    override suspend fun doWork(): Result {
        // aquí iría la lógica de sincronizar los edificios
        return Result.success()
    }
}

val pedido = PeriodicWorkRequestBuilder<SincronizacionWorker>(1, TimeUnit.HOURS)
```

```
    .setConstraints(
        Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
            .build()
    )
    .build()

WorkManager.getInstance(context).enqueue(pedido)
```

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (servicio en segundo plano que sincroniza datos y notifica el resultado mediante un BroadcastReceiver)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Sí, ambos integrantes logramos resolver completamente el ejercicio del docente siguiendo cada uno de sus pasos. Cada uno lo ejecutó en su propio emulador y obtuvo el mismo resultado: al pulsar «Sincronizar» se inicia el Service con `startService()`, que hace la tarea en segundo plano y, al terminar, avisa con un broadcast [3]. La pantalla recibe ese aviso con un `BroadcastReceiver` registrado con `RECEIVER_NOT_EXPORTED`, porque el aviso viene de nuestra propia aplicación [5]. Al desarrollarlo entendimos en la práctica cómo conectar un servicio en segundo plano con una interfaz hecha en Jetpack Compose.

A continuación se adjuntan las capturas del emulador y de Logcat de cada integrante.

Pruebas de Integrante 1

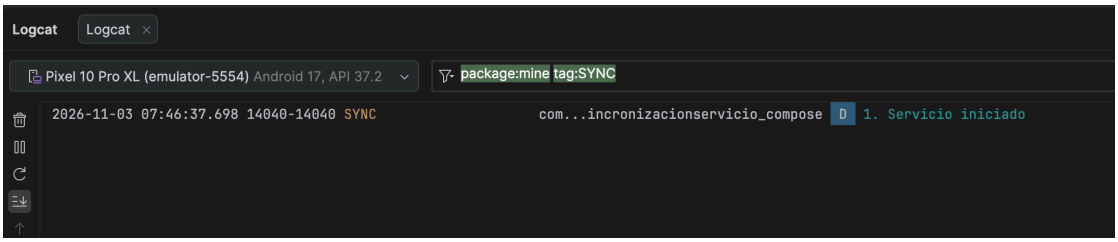


Figure 33: Evidencia 1 (Integrante 1): al presionar el botón “Sincronizar”, el emulador muestra la barra de progreso activa y en Logcat se registra “1. Servicio iniciado”.

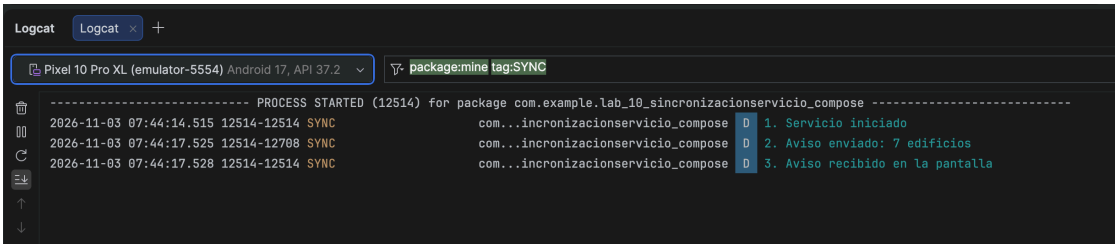


Figure 34: Evidencia 2 (Integrante 1): luego de los 3 segundos, Logcat muestra “2. Aviso enviado” y “3. Aviso recibido en la pantalla”, y la interfaz reemplaza la carga mostrando la cantidad de edificios sincronizados.

Pruebas de Integrante 2



Figure 35: Evidencia 1 (Integrante 2): al presionar el botón “Sincronizar”, el emulador muestra la barra de progreso activa y en Logcat se registra “1. Servicio iniciado”.



Figure 36: Evidencia 2 (Integrante 2): luego de los 3 segundos, Logcat muestra “2. Aviso enviado” y “3. Aviso recibido en la pantalla”, y la interfaz reemplaza la carga mostrando la cantidad de edificios sincronizados.

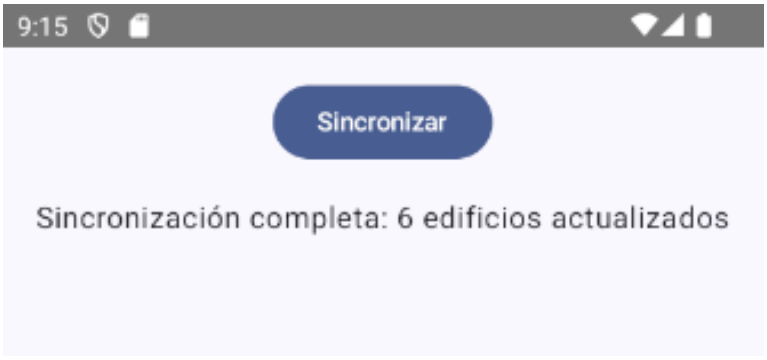


Figure 37: Evidencia 3 (Integrante 2): Sincronizacion completa“.

2. En tu propia implementación, ¿qué ocurriría si la tarea dentro de onStartCommand() se ejecutara directamente en el hilo principal en lugar de una coroutine en Dispatchers.IO? Explica el efecto sobre la interfaz de usuario.

Si en la implementación hubiéramos puesto la tarea directamente dentro de `onStartCommand()` (por ejemplo, con un `Thread.sleep(3000)`) en vez de usar una coroutine en `Dispatchers.IO`, la tarea se ejecutaría en el hilo principal, porque un Service corre ahí por defecto. Como ese hilo es el que dibuja la pantalla y atiende los toques, en la interfaz ocurriría lo siguiente:

1. **La pantalla se congelaría:** Durante los 3 segundos de espera, el hilo principal no podría dibujar nada nuevo. El indicador de progreso (`CircularProgressIndicator`) no se movería y puede que ni siquiera llegara a aparecer al presionar el botón, porque el cambio de estado no se alcanzaría a dibujar hasta que termine la espera.
2. **La aplicación no respondería al tacto:** Cualquier toque, scroll o pulsación de otro botón durante esos 3 segundos se ignoraría, y daría la sensación de que la app “se colgó”. Incluso el botón «Detener sincronización» del ejercicio propuesto no serviría.
3. **Habría riesgo de ANR:** Con solo 3 segundos probablemente no llegaría a ocurrir, pero si la tarea fuera más larga, Android muestra el aviso de “La aplicación no responde” (ANR) cuando la app no atiende un toque en 5 segundos, o cuando un Service no termina `onStartCommand()` en pocos segundos . En una sincronización real, que puede tardar más que esta simulación, esto sería un problema serio.
4. **Con la coroutine esto no pasa:** La espera se hace en otro hilo (`Dispatchers.IO`), así que el hilo principal queda libre: el indicador de progreso sigue girando y la app responde a los toques mientras se sincroniza. Por eso el ejercicio mueve la tarea a una coroutine.

III. CONCLUSIONES

- En conclusión, un `Service` no crea por sí solo un hilo secundario, sino que se ejecuta en el hilo principal de la aplicación. Por esta razón, aprendí que es indispensable combinarlo con corrutinas en `Dispatchers.IO` para procesar datos pesados de fondo sin congelar la pantalla ni generar caídas de rendimiento.
- Otra conclusión es que el `BroadcastReceiver` ayuda como un puente de comunicación entre el servicio y la interfaz en Compose de forma totalmente desacoplada. Esto evita que el servicio tenga que conocer la pantalla activa o retener referencias a la interfaz, previniendo así fugas de memoria y permitiendo que la UI reaccione naturalmente a las novedades.
- Se comprendió la importancia de gestionar responsablemente el ciclo de vida del servicio llamando a `stopSelf()` apenas se completa la tarea. Dejar procesos activos en segundo plano consume batería y memoria sin necesidad, por lo que finalizarlo a tiempo cuida los recursos del dispositivo y la estabilidad de la app.
- El uso de `DisposableEffect` junto a la directiva `RECEIVER_NOT_EXPORTED` ayudó a integrar de forma segura el receptor con Jetpack Compose. Esto permitió conectar y desconectar la escucha de datos automáticamente según el ciclo de vida de la pantalla, además de proteger la aplicación bloqueando posibles transmisiones no autorizadas desde apps externas.

Recomendaciones

- Se recomienda evaluar WorkManager si en el futuro la sincronización debe completarse incluso al cerrar la app o reiniciar el dispositivo. Esta herramienta gestiona automáticamente las restricciones del sistema, la batería y la red, evitando que el proceso sea destruido en versiones recientes de Android.
- Se sugiere también implementar cancelación cooperativa asociando la corrutina en Dispatchers.IO a un Job. De esta manera, si el usuario detiene el proceso o se llama a stopService(), la tarea de red o cálculo se cancela inmediatamente con job?.cancel(), evitando emitir eventos innecesarios.
- Otra sugerencia es evolucionar hacia StateFlow o SharedFlow dentro de un ViewModel para la comunicación interna de la app. Esto simplifica las pruebas unitarias, elimina la serialización manual en los Intent y proporciona tipado estricto en tiempo de compilación.
- Finalmente, resulta clave validar la conectividad a la red mediante ConnectivityManager antes de iniciar el servicio para no ejecutar procesos destinados a fallar. Además, conviene enriquecer la interfaz mostrando estados de progreso e información clara sobre los errores para dar más transparencia al usuario.